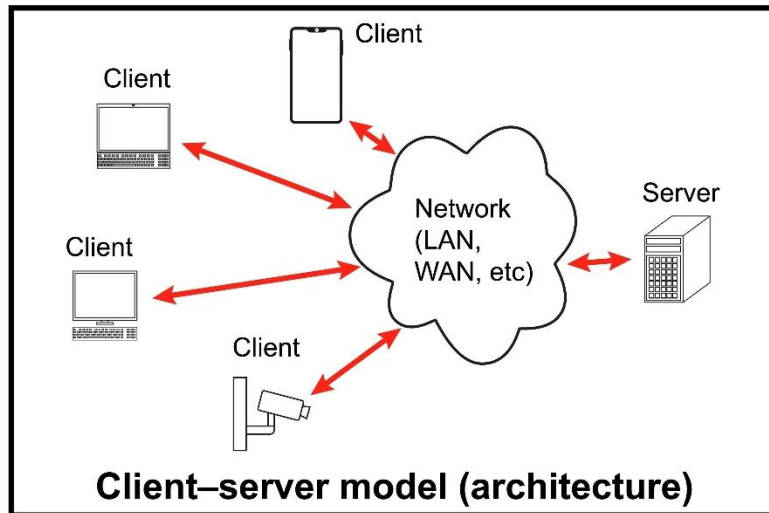


Xây dựng RESTful API Quản lý Khóa học với Expressjs



Express.js là một framework (bộ khung) nhỏ gọn và linh hoạt được xây dựng trên nền tảng **Node.js**. Nó cung cấp một bộ công cụ mạnh mẽ để phát triển các ứng dụng Web và API một cách nhanh chóng, dễ dàng hơn nhiều so với việc dùng Node.js thuần.

3 đặc điểm cốt lõi:

1. **Tối giản & Linh hoạt (Minimal & Flexible):** Express không ép lập trình viên theo một cấu trúc cứng nhắc. Nên có thể tự do tổ chức code theo ý mình.
2. **Định tuyến mạnh mẽ (Robust Routing):** Giúp chia nhỏ ứng dụng thành các đường dẫn (URL) rõ ràng như: /home, /api/users, /login... và quy định hành động cho từng đường dẫn đó (GET, POST...).
3. **Cơ chế Middleware:** Hãy tưởng tượng Middleware như một chuỗi các cửa kiểm soát an ninh. Request (yêu cầu) của người dùng sẽ đi qua từng cửa để xử lý (check đăng nhập -> check dữ liệu rác -> xử lý logic) trước khi đến đích.

Tóm lại: Express.js giúp lập trình viên Backend tập trung vào Logic nghiệp vụ (làm tính năng) thay vì phải lo lắng về các giao thức mạng phức tạp bên dưới.

Mục tiêu:

- Sinh viên tự xây dựng được Server Node.js.
- Hiểu và áp dụng đúng 4 phương thức: GET, POST, PUT, DELETE.
- Biết cách sử dụng công cụ (Postman hoặc Thunder Client) để kiểm thử API.

Giai đoạn 1: Khởi tạo dự án

Nhiệm vụ: Cài đặt thư viện và chuẩn bị môi trường.

1. Tạo thư mục:

- Mở terminal trong VSCode (trên Menu > Terminal > New Terminal), gõ:

```
mkdir lab-express-api  
cd lab-express-api
```

2. Khởi tạo Node.js:

- Lệnh: *npm init -y* (Tạo file package.json).

3. Cài đặt thư viện:

- Lệnh: *npm install express*
- Lệnh cài đặt nodemon để tự động restart server mỗi khi code thay đổi:

```
npm install --save-dev nodemon
```

4. Cấu hình script chạy:

- Mở file package.json, sửa phần "scripts":

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
}
```

Giai đoạn 2: Dựng Server & API Đọc dữ liệu (GET)

Nhiệm vụ: Tạo file server.js, khai báo dữ liệu mẫu và viết API lấy danh sách.

Code file server.js:

```
const express = require('express');  
const app = express();  
const port = 3000;  
  
// MIDDLEWARE (Rất quan trọng): Giúp server đọc được dữ liệu JSON gửi lên  
app.use(express.json());
```

```
// DATABASE GIẢ LẬP (Mảng dữ liệu)

let courses = [
  { id: 1, name: 'Lập trình Web Frontend', price: 1200000 },
  { id: 2, name: 'Lập trình Backend NodeJS', price: 1500000 },
  { id: 3, name: 'Cơ sở dữ liệu SQL', price: 900000 }
];

// --- 1. API GET: Lấy danh sách khóa học ---
app.get('/api/courses', (req, res) => {
  res.status(200).json({
    success: true,
    count: courses.length,
    data: courses
  });
});

// Khởi chạy Server
app.listen(port, () => {
  console.log(`Server đang chạy tại http://localhost:${port}`);
});
```

- **Thực thi:** Chạy lệnh `npm run dev`.
- **Test:** Mở trình duyệt truy cập `http://localhost:3000/api/courses`

Giai đoạn 3: API Thêm mới (POST)

Nhiệm vụ: Cho phép Client gửi dữ liệu lên để tạo khóa học mới. Yêu cầu phải có Validate dữ liệu.

Thêm code vào server.js:

```
// --- 2. API POST: Thêm khóa học mới ---
app.post('/api/courses', (req, res) => {
  // Lấy dữ liệu người dùng gửi lên từ body
  const { name, price } = req.body;

  // VALIDATION: Kiểm tra dữ liệu
  if (!name || !price) {
    return res.status(400).json({
      success: false,
      message: "Vui lòng nhập đầy đủ tên và giá khóa học!"
    });
  }

  // Tạo ID mới (Lấy ID cuối cùng + 1)
  const newId = courses.length > 0 ? courses[courses.length - 1].id + 1 : 1;

  const newCourse = {
    id: newId,
    name: name,
    price: price
  };

  // Lưu vào mảng
  courses.push(newCourse);

  // Trả về kết quả (Status 201: Created)
```

```
res.status(201).json({
  success: true,
  message: "Thêm thành công!",
  data: newCourse
});
});
```

- **Cách Test (Dùng Thunder Client/Postman):**

- Method: POST
- URL: `http://localhost:3000/api/courses`
- Body (JSON): `{ "name": "Python Basic", "price": 500000 }`

Giai đoạn 4: API Cập nhật (PUT) & Xóa (DELETE)

Nhiệm vụ: Tìm khóa học theo ID trên URL (:id), nếu thấy thì sửa/xóa, nếu không thấy thì báo lỗi 404.

Thêm code vào server.js:

```
// --- 3. API PUT: Cập nhật thông tin ---
app.put('/api/courses/:id', (req, res) => {
  const id = parseInt(req.params.id); // Lấy ID từ URL
  const course = courses.find(c => c.id === id);

  // Nếu không tìm thấy
  if (!course) {
    return res.status(404).json({
      success: false,
      message: `Không tìm thấy khóa học có ID = ${id}`
    });
  }
});
```

```
// Cập nhật dữ liệu (Nếu người dùng không gửi tên mới, giữ nguyên tên cũ)

course.name = req.body.name || course.name;
course.price = req.body.price || course.price;

res.status(200).json({
  success: true,
  message: "Cập nhật thành công!",
  data: course
});
});

// --- 4. API DELETE: Xóa khóa học ---
app.delete('/api/courses/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = courses.findIndex(c => c.id === id);

  if (index === -1) {
    return res.status(404).json({
      success: false,
      message: "Không tìm thấy khóa học để xóa!"
    });
  }

  // Xóa 1 phần tử tại vị trí index
  courses.splice(index, 1);

  res.status(200).json({
```

```
        success: true,  
        message: "Đã xóa thành công!"  
    });  
});
```

Giai đoạn 5: Thử thách & tổng kết

Hãy viết thêm API **Tìm kiếm khóa học**:

- Method: GET
- URL: /api/courses/search?keyword=abc
- Gợi ý: Sử dụng req.query.keyword và hàm filter() của mảng kết hợp includes().